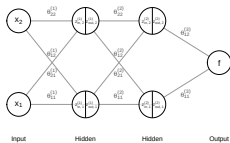
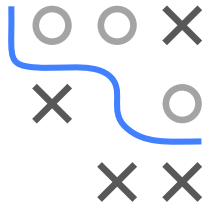


Optimization in Machine Learning

First order methods

Backpropagation and autodiff



Learning goals

- DNNs as computational graphs
- Gradients via the chain rule and vector-Jacobian products (VJPs)
- Backpropagation algorithm
- Implementation via automatic differentiation

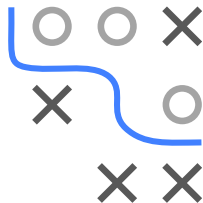
COMPUTING GRADIENTS FOR DNNs

- We now train **deep neural networks (DNNs)**: models with a very large parameter vector $\theta \in \mathbb{R}^p$ (p potentially in the millions)
- All methods discussed in this chapter rely on the gradient of the (scalar) empirical risk w.r.t. θ :

$$\nabla_{\theta} \mathcal{R}_{\text{emp}}(\theta) \in \mathbb{R}^{1 \times p}$$

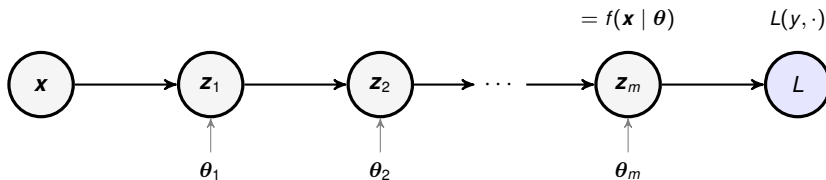
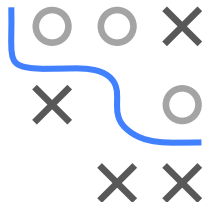
(SGD relies on mini-batch estimates of the gradient)

- For brevity, we write $L := \mathcal{R}_{\text{emp}}(\theta)$ in the following
- **Question:** How to compute it **efficiently**?



DNNs AS COMPUTATIONAL GRAPHS

- A DNN can be represented as a **directed acyclic graph** (DAG)
 - An **ordered sequence of nodes** with **directed edges**
 - Nodes hold intermediate values $\mathbf{z}_i$ according to operations $h_i(\cdot)$ of edges feeding into them
- Layer i maps $\mathbf{z}_{i-1}$ to $\mathbf{z}_i = h_i(\mathbf{z}_{i-1}; \theta_i)$
 - e.g. $\mathbf{z}_i = \sigma(\mathbf{W}_i \mathbf{z}_{i-1} + \mathbf{b}_i)$ with $\theta_i = (\mathbf{W}_i, \mathbf{b}_i)$

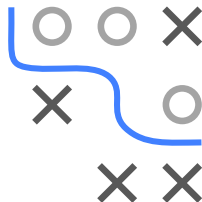


GRADIENT VIA THE CHAIN RULE

- Gradient w.r.t. the parameters θ_i factorizes via the **chain rule**:

$$\begin{aligned}\nabla_{\theta_i} L &= \frac{\partial L}{\partial \mathbf{z}_m} \frac{\partial \mathbf{z}_m}{\partial \mathbf{z}_{m-1}} \cdots \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} \frac{\partial \mathbf{z}_i}{\partial \theta_i} \\ &= \underbrace{\frac{\partial L}{\partial \mathbf{z}_i}}_{=: \bar{\mathbf{z}}_i} \frac{\partial \mathbf{z}_i}{\partial \theta_i}\end{aligned}$$

- ⇒ A complicated gradient becomes a **product of simple, local derivatives**
- ⇒ Each layer's gradient $\nabla_{\theta_i} L$ requires its associated '**node adjoint**' $\bar{\mathbf{z}}_i \in \mathbb{R}^{1 \times p_i}$



ADJOINTS AS SEQUENCES OF VJPS

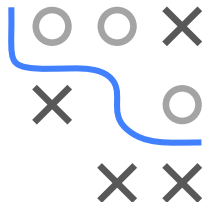
- All node adjoints $\bar{\mathbf{z}}_i$ can be recursively computed in reverse order for $i = m, (m - 1), \dots, 1$:

$$\bar{\mathbf{z}}_i = \frac{\partial L}{\partial \mathbf{z}_{i+1}} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} = \bar{\mathbf{z}}_{i+1} \frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i}$$

- $\bar{\mathbf{z}}_{i+1} \in \mathbb{R}^{1 \times p_{i+1}}$ is a (row) **vector**
- $\frac{\partial \mathbf{z}_{i+1}}{\partial \mathbf{z}_i} \in \mathbb{R}^{p_{i+1} \times p_i}$ is a **Jacobian**

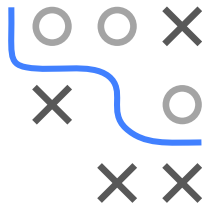
(The seed is $\frac{\partial L}{\partial L} = 1$, treated as the initial vector before $\bar{\mathbf{z}}_m = \frac{\partial L}{\partial \mathbf{z}_m}$.)

- $\Rightarrow$ This constitutes a sequence of **vector-Jacobian products (VJPs)**
- $\Rightarrow$ Each $\bar{\mathbf{z}}_i$ must only be computed **once** and can be **reused** for lower layers



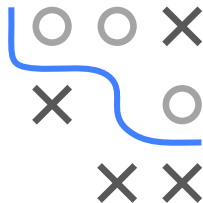
BACKPROPAGATION ALGORITHM

- This motivates the **backpropagation** algorithm to compute the full gradient $\nabla_{\theta} L$ in **two phases**:
- **Forward pass**
 - Set $\mathbf{z}_0 = \mathbf{x}$ and walk forward through the graph up to L
 - While doing so, **cache** every node value $\mathbf{z}_i$, because the derivatives in the subsequent phase are evaluated at these values
- **Backward pass**
 - Compute the adjoints $\bar{\mathbf{z}}_i$ as a sequence of VJPs by moving backward through the graph (see previous slide)
 - Obtain each parameter gradient $\nabla_{\theta_i} L = \bar{\mathbf{z}}_i \frac{\partial \mathbf{z}_i}{\partial \theta_i}$ (again as a VJP)



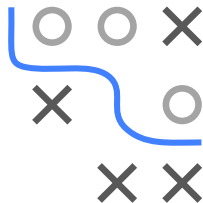
AUTOMATIC DIFFERENTIATION

- The backpropagation algorithm requires building a computational graph and computing the corresponding derivatives
 - (Reverse-mode) automatic differentiation (**autodiff**) efficiently automates this process
 - Traces code to automatically build a computational graph
 - Decomposes DNNs into **primitive operations** (e.g. $+$, $\cdot$, σ , $\exp$, $\dots$; more granular than entire layers)
 - Automatically computes derivatives
 - Applies the chain rule on the (more granular) graph and efficiently computes VJPs in the backward pass via rules (see next slides)
- ⇒ Autodiff is a key component powering modern ML frameworks (e.g. autograd in PyTorch)

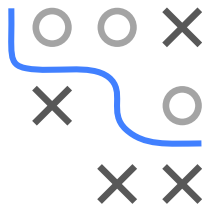


VJP RULES

- Consider a primitive operation $g : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$ and the VJP $\mathbf{v} \frac{\partial g(\mathbf{z})}{\partial \mathbf{z}} \in \mathbb{R}^{1 \times d_{\text{in}}}$
 - $\mathbf{v} \in \mathbb{R}^{1 \times d_{\text{out}}}$ as an arbitrary (row) vector
 - $\frac{\partial g(\mathbf{z})}{\partial \mathbf{z}} = \mathbf{J}_g(\mathbf{z}) \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ as the Jacobian of the operation evaluated at $\mathbf{z}$
- Autodiff stores efficient computation rules for these VJPs, i.e. $\text{vjp}_g(\mathbf{v}, \mathbf{z}, g(\mathbf{z})) = \mathbf{v} \frac{\partial g(\mathbf{z})}{\partial \mathbf{z}}$
 - Reformulation that depends only on $\mathbf{v}$ and the operation input $\mathbf{z}$ and (cached) output $g(\mathbf{z})$ $\Rightarrow$ The (potentially large) Jacobian $\mathbf{J}_g(\mathbf{z})$ need not be materialized and is only implicitly multiplied



VJP RULES: EXAMPLE



- **Example:** Consider the **elementwise sigmoid** $\sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$
 - i.e. $\mathbf{a} := \sigma(\mathbf{z})$ with $a_i = \sigma(z_i)$, each output depending only on its own input
 - The Jacobian $\mathbf{J}_\sigma(\mathbf{z}) = \frac{\partial \sigma(\mathbf{z})}{\partial \mathbf{z}} \in \mathbb{R}^{d \times d}$ is **diagonal**
 - $[\mathbf{J}_\sigma(\mathbf{z})]_{ii} = \sigma'(z_i) = a_i(1 - a_i)$ and $[\mathbf{J}_\sigma(\mathbf{z})]_{ij} = 0$ for $i \neq j$
 - Thus the VJP rule collapses to an elementwise product:
 - $\text{vjp}_\sigma(\mathbf{v}, \mathbf{z}, \mathbf{a}) = \mathbf{v} \frac{\partial \sigma(\mathbf{z})}{\partial \mathbf{z}} = \mathbf{v} \odot \mathbf{a} \odot (1 - \mathbf{a})$
- $\Rightarrow \mathcal{O}(d^2)$ cost for building, storing and multiplying the explicit Jacobian drops to $\mathcal{O}(d)$